Two-week project progress

From Tonic characterization to a generic IDXD proof seam

Hongtao Zhang

Progress window: Apr 16–30, 2026

The main shift was from "can we talk about accelerator benefit?" to a more disciplined answer: **keep Tonic claims narrow**, make the Rust IDXD path observable and maintainable, then prove a small generic DSA/IAX seam on prepared hardware.

Executive summary

Claim discipline

1. Rebuilt the Tonic comparison story around what the retained evidence actually proves.
2. Current ordinary-vs-IDXD package is a **workflow and falsification surface**, not a fresh same-host acceleration win.
3. Stronger Tonic claims still require a prepared-host rerun.

Rust path made real

1. Migrated async memmove to explicit `Bytes` / `BytesMut` ownership.
2. Implemented direct Tokio completion-record async submission.
3. Consolidated legacy `dsa-ffi` / `idxd-bindings` surfaces into `idxd-rust` + `idxd-sys`.

Generic IDXD seam

1. Added `IdxdSession<Accel>` as the shared DSA/IAX session boundary.
2. Shared the blocking lifecycle through `run_blocking_operation`.
3. Collected hardware proof and small release-profile numbers for DSA memmove + IAX crc64.

This deck deliberately separates three evidence classes: host-free contract proof, ordinary-host expected-failure classification, and prepared-host hardware proof.

Timeline: what changed in the branch

Recent work volume

Commits

136

non-merge commits since Apr 16 on this branch

Active days

6

Apr 24, 25, 27, 28, 29, 30

Core crates

3

`idxd-rust`, `idxd-sys`, `hw-eval`

Proof style

3 lanes

host-free, expected-failure, prepared-host

Milestone arcs

Apr 24–27: boundary cleanup

Tonic claim package tightened; package surfaces consolidated from stale `dsa-ffi` / `idxd-bindings` names toward canonical `idxd-rust` + `idxd-sys`.

Stop overclaiming; remove naming confusion.

Apr 27–28: async binding

Owned-buffer async API, inline ENQCMD policy, direct Tokio monitor, and benchmark/verifier surfaces landed.

Make async IDXD observable.

Apr 29–30: maintainability + generic IDXD

Readability split, lean `bon`/`snafu` convention, generic `IdxdSession<Accel>`, and representative DSA/IAX hardware proof.

Prepare handoff-worthy architecture.

Tonic claim package: honest boundary first

The current Tonic comparison evidence rejects casual acceleration claims. It proves the retained software verifier, IDXD-path verifier, and comparison-package workflow; it does **not** prove a fresh prepared-host same-host IDXD win.

Surface	Status	Meaning now
`S02 software`	**retained**	Validates curated ordinary Tonic workloads and stable artifact labels.
`S03 IDXD`	**visible gate**	Prepared-host gate for accelerated artifacts; preflight blocks remain explicit instead of hidden.
`S04 package`	**stable report**	Assembles software, IDXD, and control-floor evidence into CSV / JSON / markdown outputs.
Current ratios	**no win**	Existing `latest/` rows show IDXD at about `0.003x`–`0.653x` software throughput across curated rows.

What is safe to say

The repository can validate the ordinary path, validate the IDXD path contract, and publish a stable comparison package. The current table is useful as a blocking or falsifying surface.

What remains blocked

A stronger Tonic acceleration claim needs a prepared-host S03 pass and an S04 package regenerated from fresh live accelerated artifacts, not fixture-backed mixed provenance.

Async IDXD path: owned buffers plus direct completion

Public contract

`AsyncMemmoveRequest::new(source: Bytes, destination: BytesMut)`

The caller owns both buffers. The source length is the transfer size; the destination is returned in `AsyncMemmoveResult` with the validation report.

Rejected stale surfaces

`copy_exact`, `copy_into`, source-only constructors, public borrowed copy-back helpers, and `result.bytes` are intentionally not the post-migration API.

Runtime implementation

1. Direct Tokio runtime owns descriptors, completion records, source, destination, retry metadata, and waiter state until terminal completion.
2. ENQCMD acceptance/rejection is bounded and typed; backpressure includes retry budget/count and completion snapshot metadata.
3. The monitor resolves futures from completion snapshots and preserves accepted operation ownership even if an awaiter is dropped.
4. Hardware claim evidence remains gated by verifier `verdict=pass claim_eligible=true`.

Important design constraint: inline ENQCMD is allowed for v1; software aggregation, batching, and MOVDIR64 support are future optimization topics, not prerequisites.

Canonical package shape: less legacy ambiguity

Before consolidation

1. Active safe crate: `accel-rpc/dsa-ffi`.
2. Raw package name: `idxd-bindings` in `dsa-bindings/`.
3. Top-level `dsa-ffi/` wrapper scripts looked like another owner.
4. `tonic-profile` depended on the stale safe crate name.

After the pass

1. Canonical raw layer: `idxd-sys`.
2. Canonical safe/Tokio layer: `idxd-rust`.
3. Top-level compatibility wrappers dispatch to canonical scripts instead of defining ownership.
4. Package inventory verifier rejects active drift back to `dsa-ffi`, `idxd-bindings`, or `dsa-bindings`.

The cleanup matters because future Tonic or generic-IDXD work can now depend on one named safe layer and one named raw layer instead of rediscovering which legacy crate owns the proof surface.

Maintainability work: cleanup without semantics drift

Track	What improved	Non-change boundary
`bon` / `snafu`	Builders and SNAFU stayed where they clarify config or diagnostics: validation config, `hw-eval` config, sync/async/direct errors, and WQ-open context.	No builder around request buffers, raw descriptors, proof-private CLI structs, report schemas, or hot-loop policy.
Readability split	`idxd-rust` direct async, `tokio_memmove_bench`, `hw-eval`, and `idxd-sys` now have clearer owner modules and guard scripts.	No public API churn, schema version churn, raw unsafe hiding, or prepared-host claim from host-free proof.
`idxd-sys` raw boundary	Descriptor, portal, completion, timing, topology, and cache concerns are separated behind a lean facade.	Raw ABI layout, `std::io::Result`, volatile status reads, and ENQCMD accepted/ rejected signals stay visible.

Why this was worth doing

Most of the next risk is integration complexity, not one missing helper. Making owner boundaries visible reduces the chance that future work duplicates lifecycle code or overstates ordinary-host checks.

Proof discipline preserved

Reports and verifiers keep the no-payload rule: diagnostics may include paths, lengths, phases, statuses, retry counts, and timings, but not source/destination bytes or dumps.

Generic IDXD architecture: one seam, representative operations

Core architecture

1. `IdxdSession<Accel>` owns one `WqPortal` and one typed config.
2. `Dsa`, `Iax`, and `Iaa` keep accelerator-family naming explicit.
3. `IdxdSession<Dsa>::memmove` and `IdxdSession<Iax>::crc64` are the representative public operations.
4. Static marker types avoid public `dyn` dispatch or a broad operation hierarchy.

Shared lifecycle

1. `run_blocking_operation` owns reset, fill, submit, observe, classify, retry, and return.
2. DSA and IAX adapters own descriptor/completion state and operation-specific classification.
3. `WqPortal::submit_desc64` owns the dedicated/shared 64-byte descriptor submission branch.
4. Proof and benchmark CLIs consume the API instead of bypassing it.

The elegance decision is intentionally scoped: no full DSA/IAX surface, no scheduler, no pooling, no batching framework, no RPC integration, and no benchmark matrix inside M011.

Prepared-host proof and measured rows

Evidence	Target	Device	Bytes	Iters	Mean latency	Rate
S03	dsa-memmove	/dev/dsa/wq0.0	64	n/a	completed	verdict=pass
S03	iax-crc64	/dev/iax/wq1.0	64	n/a	completed	crc64_verified=true
S04	dsa-memmove	/dev/dsa/wq0.0	4096	1000	6,837 ns	146,246 ops/s
S04	iax-crc64	/dev/iax/wq1.0	4096	1000	2,178 ns	459,064 ops/s

Verifier

verdict=pass

S03 operation proof and S04 measured-number proof both passed

Profile

release

S04 required release-profile measurements with `claim_eligible=true`

Failures

0 / 2000

S04 representative benchmark completed all required operations

These are representative proof rows, not a performance characterization: no size sweep, concurrency sweep, batching comparison, software baseline, or optional shared-DSA claim is included.

What is now stronger than two weeks ago?

Supported now

1. The canonical Rust IDXD stack has a real safe layer and raw layer with host-free guards.
2. Direct Tokio async memmove has explicit buffer ownership, completion-driven futures, and typed failure metadata.
3. Generic DSA/IAX session architecture is not just sketched: representative DSA memmove and IAX crc64 completed on prepared hardware.
4. Release-profile representative rows prove the new seam can produce positive measured metrics.

Still not supported

1. A Tonic end-to-end IDXD speedup claim.
2. Full DSA/IAX/IAA operation coverage.
3. A production scheduler, pooling layer, batching policy, or MOVDIR64 strategy.
4. Broad benchmark conclusions from the tiny representative S04 proof.

The progress is architectural credibility plus proof plumbing: the project can now make small hardware-backed claims honestly, while rejecting bigger claims until the matching evidence exists.

Recommended next discussion

Option A — Tonic rerun

Use the prepared-host path to refresh the IDXD subtree and rebuild the ordinary-vs-IDXD package.

Best if the meeting needs an application-level claim.

Option B — generic seam expansion

Add the next DSA or IAX operation only when a concrete consumer and verifier exist.

Best if architecture maturation is the goal.

Option C — proof utility extraction

Wait for one more peer verifier, then extract shared launcher/artifact/no-payload helpers.

Best if verifier duplication starts slowing work.

My recommendation: start with **Option A** if the advisor meeting is about the original Tonic motivation; choose **Option B** only if the next milestone is explicitly about the IDXD library surface.